Assignment 1

Exercise groups 10

Magnus Fidje, Oskar Støre and Håkon Skramstad

Exercise 1

In [13]:
```python
# Insertion sort algorithm from lecture
# Added print statments for each repetion of the loop
# Also used 1-based indexing to match pseudocode
A = [8, 9, 2, 3, 7, 1]

def InsertionSort(A):
    print(" j  |  i  | key  | A")
    print("--------------------------------")
    for j in range(1, len(A)):
        key = A[j]
        i = j - 1
        print(f" {j+1}  |  {i+1}  |  {key}   | {A}")
        while i >= 0 and A[i] > key:
            A[i + 1] = A[i]
            i -= 1
        A[i + 1] = key
    print(f" Terminated      | {A}")
    print("--------------------------------")
    return A

b = InsertionSort(A)
```

```
 j  |  i  | key  | A
--------------------------------
 2  |  1  |  9   | [8, 9, 2, 3, 7, 1]
 3  |  2  |  2   | [8, 9, 2, 3, 7, 1]
 4  |  3  |  3   | [2, 8, 9, 3, 7, 1]
 5  |  4  |  7   | [2, 3, 8, 9, 7, 1]
 6  |  5  |  1   | [2, 3, 7, 8, 9, 1]
 Terminated      | [1, 2, 3, 7, 8, 9]
--------------------------------
```

Exercise 2

Psuedocode

```
    SequentialLinearSearch(A, n, v):
        last = A[n]
        A[n] = v
        i = 1

        while A[i] != v do
        i = i + 1

        A[n] = last

        if i < n or last = v then
            return i
        else
            return NULL
```

Run time

Best case run time: 1

Worst case runce time: n

Average run time: n

Exercise 3

1

It would be an advantage to use numbers as sort keys instead of sorting the strings because it saves up much more time. For a computer it's cheaper to sort numbers than strings, because numbers have a fixed size compared to strings than can be variabel size, and you may have to go through the whole string.

2

The numbers created must fulfill the condition of being sortable. Each letter must be mapped to a number, where A is smaller than B, B is smaller than C and so on

3

For converting strings into numbers, each string gets assigned the ASCI value. Save each strings as a list, and if the name is shorter then 8 bytes, fill the remaining bytes with zeroes.

Exercise 4

Insertion sort is faster than merge sort for n < 256

Exercise 5

The statment at least $O(n^2)$ is meaningless, becuase O(n) notation is always upperbound. And in most cases you need to do more than lowerbound.

Exercise 6

The function T(n) = 2 n^3 - 5 $\sqrt{n}$ + 4n $\lg n$ in Θ notation is:

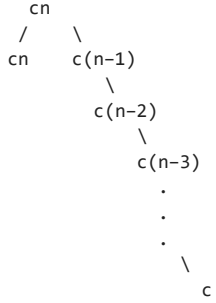
$$T(n) = \Theta(n^3)$$

Exercise 7

In [20]:
```python
# Task 1, recursive algorithm
def mini(A):
    if len(A) == 0:
        return None
    if len(A) == 1:
        return A[0]
    min_value = mini(A[1:])
    return A[0] if A[0] < min_value else min_value
```

2

Recursion tree for the algorithm, and estimate run time T(n)



3

Find a recurrence equation for T(n):

$$T(n) = \begin{cases} \Theta(1) & \text{if } n \leq c, \\ T(n-1) + \Theta(1) & \text{otherwise} \end{cases}$$

Task 4

Compare runtime for the loop-based algorithm versus the recursive algorithm

In [ ]:
```
A = [3,2,1,5,3,4,3,4,23,4,3,53,5,4,4,5,3,34,3,3,454,3, -1]

def minium(data):
    min_value = data[0]
    for next_value in data:
        if next_value < min_value:
            min_value = next_value
    return min_value
```

Best case

In [22]:
```
%timeit mini(list(range(100)))
```

```
27.6 µs ± 1.13 µs per loop (mean ± std. dev. of 7 runs, 10,000 loops each)
```

In [23]:
```
%timeit minium(list(range(100)))
```

```
2.39 µs ± 35 ns per loop (mean ± std. dev. of 7 runs, 100,000 loops each)
```

Worst case

In [24]:
```
%timeit mini(list(reversed(range(100))))
```

```
29.3 µs ± 2.52 µs per loop (mean ± std. dev. of 7 runs, 10,000 loops each)
```

In [25]:
```
%timeit loop(list(reversed(range(100))))
```

```
4.27 µs ± 239 ns per loop (mean ± std. dev. of 7 runs, 100,000 loops each)
```

The compecity is the same for both algorithms ($\Theta(n)$), but because it's slower to do a function call than iterating a loop in Python, there is substantial time difference in execution time.